

DAY 3 · 수요일

실생활 미니 프로젝트 + 리서치 이론 · 팀 구성

도구를 실제 문제에 적용하고, 리서치 프로세스를 이해한 뒤 팀과 주제를 정한다

담당 세션 (3)

오전 특강

10:00-11:30 · 90 분

미니 프로젝트 기획 (개인)

오후 특강 1

13:00-14:20 · 80 분

미니 프로젝트 개발 (개인)

오후 특강 2

14:30-16:00 · 90 분

리서치 프로세스 · 쟁점 + 팀 구성

전체 과정 로드맵

DAY 1

월요일

LLM 원리와 이해

DAY 2

화요일

에이전트 원리 + 클로드
코드

DAY 3

수요일

미니 프로젝트 + 리서치
이론 · 팀

● 진행 중

DAY 4

목요일

팀 리서치 — 기획 · 구현 ·
실행

DAY 5

금요일

실행 · 분석 + 보고서 ·
발표

진행 단계 : 이론 → 도구 → 실전 → 결과 → 발표

세션 구성 (3)

1

AI Agent 기반 미니 프로젝트 기획 (개인)

10:00-11:30

오전 특강 · 문제 정의 · 범위 · 산출물 설계

90 분

2

AI Agent 기반 미니 프로젝트 개발 (개인)

13:00-14:20

오후 특강 1 · 바이브코딩으로 직접 구현

80 분

3

리서치 프로세스 (기존 · agentic) + 쟁점 + 팀 · 주제

14:30-16:00

오후 특강 2 · 생성 vs 선별 토론 + 팀 · 주제 · corpus 확정

90 분

오전 특강 10:00-11:30 · 90 분

AI Agent 기반 실생활 미니 프로젝트 — 기획 (개인)

작고 실용적인 개인 프로젝트를 선정하고, 구현 전에 설계한다.

AI Agent 기반 실생활 미니 프로젝트 — 기획 (개인)

이 세션에서 다룰 내용

1 왜 기획이 먼저인가

2 좋은 미니 프로젝트의 조건

3 문제를 한 문장으로

4 기획 틀

5 기획서 작성 (실습)

왜 기획이 먼저인가

- **방향 없는 구현은 헤맨다**
기획이 절반
- **범위를 좁혀야 끝낼 수 있다**
한 세션 안에 결과
- **'완성'의 정의를 먼저**
산출물 합의

좋은 미니 프로젝트 고르기

- 작고 명확하게

짧은 시간에 결과가 보이는 범위

- 실생활 문제에서 출발

예 : 가계부 요약 · 일정 정리 · 자료 검색 도우미

- 도구로 실현 가능한가

현실성 점검

문제를 한 문장으로

- 누구의 어떤 불편인가

문제 진술

- 왜 지금 풀 가치가 있나

지금 불편이 크거나 자주 생기나

- 성공하면 무엇이 달라지나

전 · 후 차이를 한 줄로

기획 시 정할 것

문제 / 사용자

- 해결할 문제
- 누가 쓰나
- 성공 기준
- 사용 맥락

범위 / 산출물

- 할 것 / 안 할 것
- 필요한 도구
- 결과 형태
- 마감

개인 프로젝트 기획서 작성

실습 단계

- 1 문제 한 문장 정의
- 2 사용자 · 성공기준 적기
- 3 범위 정하기
- 4 산출물 형태 결정

산출물

한 장짜리 기획서

템플릿 :
examples/mini-project/project-plan-
template.md

오후 특강 1 13:00-14:20 · 80 분

AI Agent 기반 실생활 미니 프로젝트 — 개발 (개인)

기획을 바이트코딩으로 구현하며 도구 사용에 익숙해진다 .

AI Agent 기반 실생활 미니 프로젝트 — 개발 (개인)

이 세션에서 다룰 내용

1 바이브코딩 진행 원칙

2 개발 실습

3 회고

바이트코딩으로 만들기

- 의도를 명확히 전달

좋은 프롬프트 = 좋은 결과

- 작은 단위로 반복

만들고 → 확인하고 → 고치고

- 검증을 습관화

동작을 눈으로 확인

개인 프로젝트 개발 실습

실습 단계

- 1 기획서 기반으로 시작
- 2 기능 단위로 구현 요청
- 3 실행 · 검증 · 수정 반복
- 4 결과 정리

산출물

동작하는 미니 프로젝트
(완성 시 1 분 데모)

주요 학습 내용

- **도구의 강점과 한계 체감**

다음 단계로의 교훈

- **리서치 에이전트로 확장**

다음 세션 연결

오후 특강 2 14:30-16:00 · 90 분

리서치 프로세스 (기존 · agentic) + 쟁점 토론 + 팀 · 주제 설정

기존 연구 프로세스와 그 agentic 버전 · 핵심 쟁점을 함께 본 뒤 , 팀과 리서치 주제를 정한다 .

리서치 프로세스 (기존 · agentic) + 쟁점 토론 + 팀 · 주제 설정

이 세션에서 다룰 내용

1 API 란 무엇인가 (바이트코딩에서 만난 개념)

2 현대 에이전트 공학 (하네스 · 루프 · 컨텍스트)

3 기존 연구 프로세스

4 agent 로 옮길 때 4 가지 보정

5 agentic 5 개 질문 (①~⑤ 상세)

6 핵심 쟁점 : 생성 vs 선별

7 권고 · 단계적 선별 · 신약 유의점

8 팀 적용 토론

9 프로젝트 형식 · 파이프라인

10 팀 구성 · 기획 (실습)

프로그램끼리 대화하는 '창구'

- **무엇**

Application Programming Interface — 한 프로그램이 다른 프로그램의 기능·데이터를 요청하는 정해진 창구

- **비유**

식당의 메뉴판·주문대 — 주방 (내부) 을 몰라도 정해진 방식으로 주문하면 결과가 나온다

- **핵심**

내부 구현은 감춰지고 , ' 무엇을 어떻게 요청하는지 ' 라는 약속만 공개된다

- **왜 생소했나**

바이브코딩이 만든 코드가 외부 서비스 (PubMed·LLM 등) 를 이 창구로 불러 썼기 때문

NOTE API 는 남이 만든 기능을 내 코드에서 빌려 쓰기 위한 표준화된 약속이다 .

요청 → 응답, 그리고 키

- 요청 (request)

'무엇을 달라'를 정해진 형식으로 보낸다 — 예: '이 검색어로 논문 20 편'

- 응답 (response)

상대 서버가 결과를 정해진 형식 (주로 JSON) 으로 돌려준다

- API 키 (key)

'누가 요청하는지' 식별하는 비밀번호 — 사용량·권한 관리에 쓰인다

- 예

collect_pubmed.py 가 PubMed 에 검색을 요청하고 논문 목록을 받아온 것

NOTE 대부분의 API 는 '요청을 보내면 정해진 형식으로 응답이 온다' 는 단순한 구조다 .

API 를 어디서 만나나

- 논문 수집

PubMed API 로 검색어에 맞는 논문을 자동으로 가져온다

- LLM 사용

Claude 같은 모델도 API 로 호출된다 — 앱 · 스크립트가 모델에 요청을 보낸다

- 에이전트의 도구

어제 배운 '도구' 의 상당수가 실은 외부 API 호출이다

- 주의

키는 비밀번호처럼 다뤄 노출하지 않는다 — 코드 · 공개 저장소에 넣지 않기

NOTE API 는 에이전트가 바깥 세계 (데이터 · 모델 · 서비스) 와 연결되는 통로이며 , 우리 실습 곳곳에서 쓰인다 .

왜 ' 모델 다음 ' 을 설계하는가

- **모델 향상의 한계**

한 번의 호출을 더 똑똑하게 만드는 여지는 점점 줄어든다

- **작업은 한 번에 끝나지 않는다**

복잡한 일에는 도구 사용 · 여러 단계 · 검증이 필요하다

- **레버리지의 이동**

성능은 이제 모델 자체보다 ' 모델을 둘러싼 설계 ' 에서 갈린다

- **예**

같은 모델이라도 Claude Code 처럼 도구 · 반복을 갖춘 환경에서 훨씬 큰 일을 해낸다

NOTE 프롬프트 한 줄을 잘 쓰는 단계를 넘어 , 모델이 일하는 환경 전체를 설계하는 단계로 이동했다 .

프롬프트에서 하네스 · 루프로

- **프롬프트 엔지니어링**

한 번의 질문을 잘 쓰는 일 — 복잡한 작업에는 부족하다

- **컨텍스트 엔지니어링**

한정된 맥락 창에 '무엇을 넣을지' 설계 — 검색 · 요약 · 메모리

- **하네스 엔지니어링**

도구 · 파일 · 환경 · 권한 등 모델 주변 골격을 설계

- **루프 엔지니어링**

관찰→사고→행동 반복 구조 자체를 설계

NOTE 네 가지는 서로 대체하지 않고 누적된다 — 뒤로 갈수록 모델 '바깥'을 더 많이 설계한다.

모델을 ' 에이전트 ' 로 만드는 골격

- **무엇**

모델이 행동하도록 감싸는 도구 · 환경 · 권한의 집합 (harness= 마구)

- **어떻게**

검색 · 파일 읽기 / 쓰기 · 코드 실행 · 하위 에이전트 등 행동 수단을 연결한다

- **왜 중요**

모델 자체는 그대로여도 , 무엇을 줄 수 있느냐가 할 수 있는 일의 범위를 정한다

- **예**

Claude Code = LLM 에 터미널 · 파일 · 실행 도구를 붙인 하네스

NOTE 어제 배운 'LLM + 도구 + 반복 = 에이전트' 에서 '도구' 를 설계하는 일이 하네스 엔지니어링이다 .

반복 · 검증 구조의 설계

- 무엇

관찰→사고→행동을 한 번이 아니라 여러 번 돌리는 구조를 설계한다

- 어떻게

행동의 결과를 다시 관찰해 다음 행동을 정하고, 언제 멈출지 (stop) 기준을 둔다

- 검증 내장

각 반복에 '스스로 점검'을 넣어 오류를 다음 단계에서 교정한다

- 예

코드를 고치고 → 실행해 보고 → 실패하면 다시 고치는 자동 반복

NOTE 한 번에 완벽한 답을 기대하지 않고, 반복과 자기 점검으로 품질을 끌어올리는 것이 핵심이다.

한정된 맥락 창 의 운용

- **무엇**

모델이 한 번에 보는 맥락 (context window) 은 유한하다 — 무엇을 넣을지 고른다

- **어떻게**

필요한 자료만 검색 (RAG) 해 넣고 , 길어지면 요약 · 압축하며 , 메모리 파일로 핵심을 유지

- **왜**

맥락이 정확 · 간결할수록 환각이 줄고 일관성이 올라간다

- **예**

CLAUDE.md 에 프로젝트 규칙을 적어 매번 같은 맥락을 주입한다

NOTE 무엇을 넣느냐만큼 ' 무엇을 빼느냐 ' 도 설계의 일부다 .

우리 리서치 에이전트도 하네스 · 루프 설계

- 하네스

PubMed 수집 · 파일 정리 · 평가 등 행동 수단을 갖춘다

- 루프

자료파악 → gap → 가설생성 → 평가를 반복하며 다듬는다

- 컨텍스트

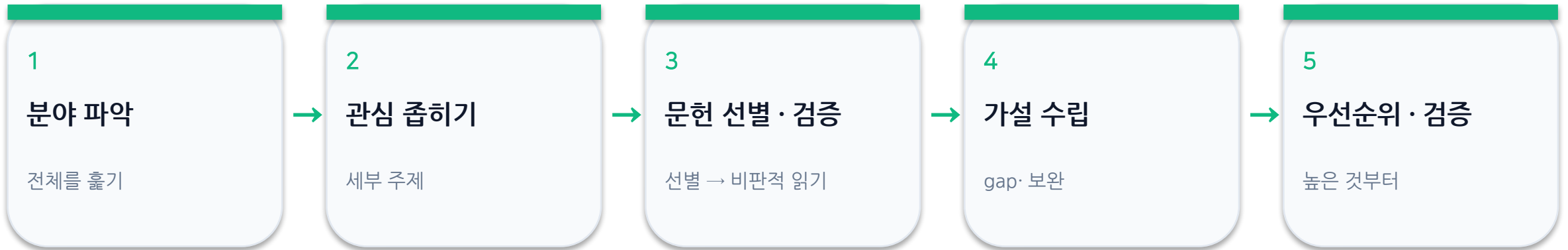
수집한 논문을 같은 틀로 구조화해 핵심만 맥락에 올린다

- 그래서

이제 볼 '연구 프로세스'는 이 세 가지를 연구에 맞게 설계하는 일이다

NOTE 다음에 다룰 리서치 프로세스는 추상적 이론이 아니라, 방금 본 현대 에이전트 공학을 연구에 적용하는 구체적 설계다.

기존 연구 프로세스 (사람)



NOTE 실제 연구 과정은 한 방향으로 진행되지 않고 반복적이며, 비용이 큰 '재현' 보다 주장과 근거를 점검하는 '검증'에 가깝다.

agent 로 옮길 때 — 4 가지 보정

- 선형 → 반복 루프

가설 · 검증 결과가 '어떤 문헌이 중요한가' 를 다시 바꾼다

- '재현' → '검증'

전체 재현은 비용상 어려움 → 주장 · 근거 점검 , 가정 · 한계 식별로 재정의

- '질문 정의' 와 'gap 분석' 을 명시

연구의 단위는 분야가 아니라 질문이며 , 가설의 최대 원천은 문헌 간 모순 · 공백이다

- feasibility 를 1 급 제약으로

가용 데이터 · 도구로 테스트 가능한 가설만 생성하도록 처음부터 제약

NOTE 기존 골격은 타당하되 , agent 로 자동화하려면 이 네 가지를 반드시 보정해야 한다 .

각 단계를 agent 로 — 5 개 질문

- ① 분야 파악

전체를 어떻게 파악할 것인가

- ② 논문 선별

좋은 논문을 어떻게 고를 것인가

- ③ 논문 학습

어떻게 깊이 이해할 것인가

- ④ 가설 도출

개선 · 새 가설을 어떻게 만들 것인가

- ⑤ 가설 우선순위

무엇부터 검증할 것인가

NOTE 이 다섯 단계가 리서치 에이전트가 자동화하려는 대상이며, 이제 하나씩 살펴본다.

분야 파악 — 전부 읽지 않는다

- **출발**

리뷰 논문과 대표 (landmark) 논문에서 시작 — 분야가 스스로 압축해 둔 지도

- **방법**

LLM 의 기존 지식을 뼈대로 삼되 , 검색 (RAG) 으로 최신성과 사실을 교정

- **산출물**

하위 분야 지도 (taxonomy) · 핵심 쟁점 · 개념 · 인용 관계도

- **멈추는 시점**

새로운 정보가 거의 나오지 않는 지점 (saturation)

NOTE 목표는 분야를 전부 아는 것이 아니라 , 이해결 지점과 모순을 찾는 것이다 .

논문 선별 — 무엇을 기준으로

- 선별은 필요한가

필요하다 — 단 사람은 '읽을 시간', 에이전트는 '신호 품질' 이 병목

- 절차

넓게 수집 → 초록으로 1 차 선별 → 순위화 → 상위 + 모순 · 예외 논문만 정독

- 기준

주제 적합도와 다양성 (반대 견해 포함)

- 주의

인용 수는 하나의 참고 지표일 뿐 — 시간이 지나야 쌓이고 편향이 있다

NOTE 유망 논문만 좁히기보다 모순 · 예외를 일부러 남겨 가설의 단서를 확보한다 .

논문 학습 — 읽기가 아니라 구조화

- 핵심

같은 틀 (schema) 로 핵심 정보를 추출 — 문제 · 방법 · 데이터 · 주장 · 근거 · 가정 · 한계

- 교차 구조

주장 간 지지 · 반박 관계 , 방법의 계보 , 상충 지점을 연결

- 왜

가치는 개별 논문이 아니라 ' 관계 구조 ' 에 있으며 , 그 안에서 gap 이 드러난다

NOTE 정규화된 비교가 가능해야 문헌 간 공백과 모순이 보인다 .

가설 도출 — 구조에서 가설로

- **핵심 원칙**

'창의적으로'가 아니라 '이 구조적 단서에서' 만든다 — 검증 가능 · 환각 감소

- **생성 방식**

모순 · 공백 / 조합 · 전이 / 가정 완화 / 규모 확장 / 실패 원인 보완

- **근거 태그**

각 가설에 어느 gap·논문에서 나왔는지 출처를 부착

NOTE 이어지는 슬라이드에서 주요 생성 방식을 하나씩 살펴본다.

모순 · 공백에서

- 모순

두 논문의 주장이 충돌하는 지점 — '왜 다른가'가 가설이 된다

- 미검증

주장은 있으나 근거·재현이 약한 지점

- 빠진 관점

특정 집단·조건·조합이 다뤄지지 않은 지점

- 예

A 약물이 한 연구에선 유효, 다른 연구에선 무효 → '특정 유전형에서만 유효할 것이다'

NOTE 문헌 간 모순은 가장 비옥한 가설의 원천이다.

조합 · 전이

- 조합

서로 다른 논문의 아이디어를 결합해 새 가설을 만든다

- 전이

A 분야에서 검증된 방법 · 지표를 B 문제에 적용한다

- 연결

Day 4 의 MOOSE-Chem 영감 (inspiration) 방식과 동일한 발상

- 예

한 질환에서 검증된 바이오마커를 다른 질환의 조기 진단에 적용하면 ?

NOTE 비용이 낮으면서도 새로움이 큰 가설을 얻기 좋은 방식이다 .

가정 완화 · 규모 · 실패 보완

- 가정 완화

논문이 전제한 가정 · 한계를 풀면 어떻게 되는가

- 규모 · 집단 확장

소규모 · 특정 집단의 결과를 더 크거나 다른 집단으로

- 실패 원인 보완

보고된 한계 · 실패 (예 : 독성) 를 회피하도록 설계

- 예

성인 대상 결과를 노인의 약물동태 (PK) 차이를 고려해 재검증하면 ?

NOTE 기존 연구의 경계와 실패 지점이 새로운 질문을 만든다 .

좋은 가설을 평범한 추측과 구분

- 검증 가능

'~ 한 조건에서 ~ 할 것이다' 라는 한 문장으로 표현

- 근거 부착

어느 gap·논문에서 나왔는지 출처 태그를 단다

- 환각 방지

근거 없는 가설은 제외 — 반드시 구조적 단서에 기반

NOTE 검증 가능성과 근거 추적성이 좋은 가설의 핵심 조건이다.

가설 우선순위 — 무엇을 먼저

- 필요성

가설을 많이 만들었다면, 무엇을 먼저 검증할지 정해야 한다

- 점수 공식

기대 영향 (impact) × 타당성 (plausibility) ÷ 테스트 비용

- 게이트

검증 가능성 (feasibility) 과 새로움이 낮으면 제외

NOTE 이어지는 슬라이드에서 평가 기준과 그 한계를 살펴본다.

무엇으로 점수를 매기나

- 기대 영향 (impact)

맞다면 분야에 얼마나 중요한가

- 타당성 (plausibility)

기존 근거에 비추어 그럴듯한가

- 테스트 비용

검증에 드는 시간·자원 — 낮을수록 우선

- 새로움

이미 연구된 것은 아닌지 문헌과 대조

NOTE 영향과 타당성은 높고 비용은 낮은 가설을 상위로 올린다.

LLM 의 순위 판단을 그대로 믿지 말 것

- 문제

새 가설의 순위 판단은 LLM 이 불안정하고 , 그럴듯한 쪽으로 치우친다

- 보정 ①

여러 번 · 여러 관점으로 평가해 합의 (ensemble) 를 본다

- 보정 ②

모델 판단보다 비용이 낮은 경험적 사전 점검을 우선한다

- 보정 ③

단일 최선이 아니라 후보군 (portfolio) 을 유지한다

NOTE 우선순위 자체가 불확실하므로 , 미리 하나로 좁히지 않고 후보군으로 대비한다 .

가설 : ' 다수 생성 ' vs ' 선별 '

다수 생성 (breadth)

- 가용 데이터 / 도구로 제약
- 싼 테스트로 거르기
- 다양성↑ · LLM 판단 의존↓

사전 선별 (selection)

- 비싼 테스트 전에 추리기
- LLM 판단에 의존
- 위양성↓ · 놓칠 위험

왜 '다수 생성 + 싼 필터'에 무게를 두나

- LLM의 최약점

새 가설의 품질을 미리 (a priori) 판단하는 일 — 환각 · 과신이 큼

- 그래서

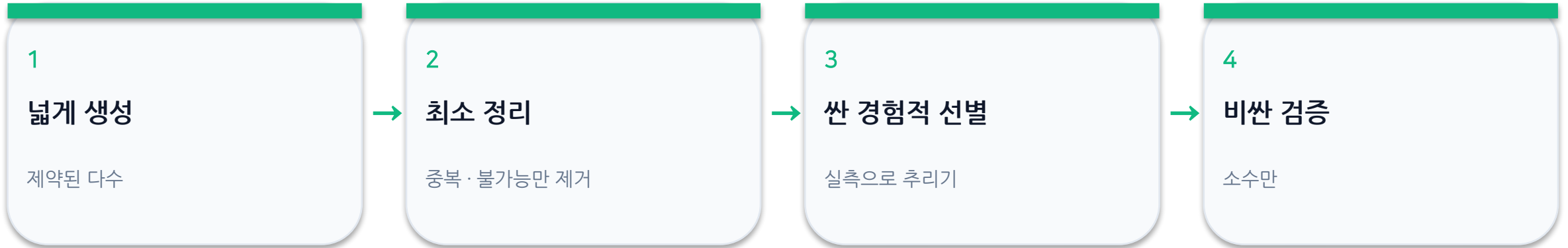
현실에 근거한 싼 테스트로 거르는 편이 '똑똑한 선별'보다 신뢰할 만하다

- 핵심 enabler

가용 데이터 · 도구로 제약 → 싼 테스트를 실제 가능케 하고 탐색 공간을 묶는다

NOTE 테스트 비용이 싸지는 조건에서는 무게중심이 '생성' 쪽으로 이동한다.

생성에서 검증까지 — 단계적 선별



NOTE 선별은 모델의 판단이 아니라 비용이 낮은 실측으로 수행하며, 유망 가설을 미리 단정하지 않는다.

유의점 — 특히 신약 분야에서

- 다중검정 위양성

많이 테스트하면 우연한 '성공' 이 늘어남 → hold-out·재현·통계 보정 필요

- 싼 테스트는 proxy

in-silico· 문헌은 진짜 결과의 대리물 — 대리지표만 좋아지는 Goodhart 주의

- 신약·임상은 단계적

최종 검증 (결합·동물·임상) 은 agent 로도 안 싸짐 → 싼 in-silico 로 넓게 → 좁히고 → 비싼 실측

NOTE 신약 분야에서 'agent 가 싸게 하는 것' 은 in-silico· 문헌 단계임을 분명히 인식해야 한다 .

우리 팀 주제에 적용

- 어디에 무게를 ?

생성 (많이) vs 선별 (미리 추리기) — 팀 분야에선 ?

- 싼 테스트가 가능한가 ?

우리 데이터 · 도구로 가설을 어떻게 빠르게 거를까

- 제약 조건은 ?

어떤 데이터 · 도구 범위로 가설을 묶을까

NOTE 정답을 정하기보다 , 팀 분야의 조건에 비추어 전략을 선택하는 토론이다 .

프로젝트 형식 + 3 세션 스코프

- **무엇**

자기 관심 분야에서 'rationale 갖춘 새 가설' 을 발견 · 제안하는 에이전트

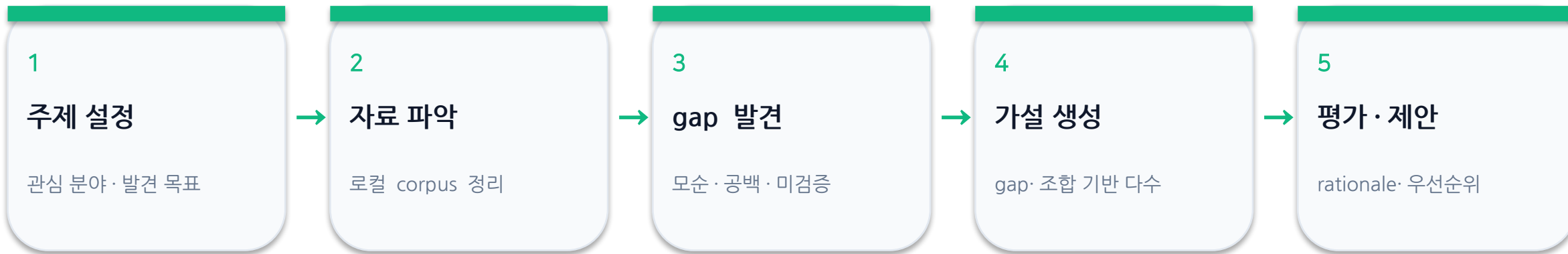
- **스코프**

corpus 기반 · 자료파악→ gap 발견→가설생성→평가 · 제안 · Day 4 안에 완주

- **산출물**

rationale 갖춘 새 연구 가설 제안서 (Q&A 보고서 아님)

가설 생성 파이프라인 (만들 것)



NOTE 앞서 살펴본 agentic 5 단계와 동일한 흐름이며, 4 일차에 직접 구현하고 실행한다.

팀 구성 & 분야 · 기획

실습 단계

- 1 팀 편성 · 역할 분담
- 2 관심 분야 + 발견 목표 (무엇을 제안하고 싶나)
- 3 자료 (corpus) 확보 계획
- 4 팀 기획서 작성

산출물

팀 기획서
(분야 · 발견 목표 · corpus · 산출물 =
가설 제안 · 스코프)
예 : 신약 / 바이오 리서치 등
템플릿 :
examples/research-agent/research-
plan-template.md

핵심 정리 및 다음 과정

핵심 정리

- 우리 목표 = corpus 의 gap 에서 'rationale 갖춘 새 가설' 을 발견 · 제안 (질문 답변이 아님)
- 가설 : 테스트가 싸지면 ' 제약된 다수 생성 + 싼 필터 ' 가 유리 — 단 proxy · 위양성 주의
- 팀 · 분야 · 발견 목표 · corpus 계획 확정 — Day 4 에 가설 생성 에이전트 구현

다음 과정 — Day 4

확정한 분야로 corpus 기반 가설 생성 에이전트를 Day 4 3 세션 안에 구현 · 실행해 새 가설을 제안합니다 .